Computer Graphics

CSCI 490J, CSCI 631

Project 2

The Graphics Pipeline - Transformations and Simple Objects

Summary

The purpose of this assignment is to implement the graphics transformation pipeline and routines for drawing 3D primitives.

Program Requirements

Your program should be able to

- draw 3D points, lines, cubes, spheres, cones, disks, arbitrary point sets, and arbitrary polygon sets as wireframe objects.
- properly deal with arbitrary camera parameters
- properly clip points in x, y and z with arbitrary near and far clipping planes.

Several scene files are available for this project. Each set will progressively test various aspects of your pipeline.

Things That Ought to be Implemented

This is a list of data structures and routines that need to be implemented. This is not an inclusive list. You may need to implement other routines.

Functions

The functions listed below are **not** necessarily the function headers you should use. But somehow, you should have the functionality for the following:

Transformations

- `identity(xform return)` *or* `identity(xform m1)` -- The first returns an identity matrix. The second clears the matrix passed in to the identity.
- `multiply(xform return, xform m1, xform m2)` -- Multiplies m1 by m2 (m2 on the right) and returns the resulting transformation.
- `multiply(pointh return, xform m1, pointh p1)` -- Multiplies m1 by p1 on the right and returns the resulting homogeneous point.
- `translate(xform m1, double tx, double ty, double tz)` -- Changes m1 by multiplying on the right by a translation matrix formed by tx, ty, and tz.
- `scale(xform m1, double sx, double sy, double sz)` -- Changes m1 by multiplying on the right by a scale matrix formed by sx, sy, and sz.

- `rotate_xy(xform m1, double theta)` -- Changes m1 by multiplying on the right by a rotation matrix using rotation of x towards y. Theta is given in degrees.
- `rotate_yz(xform m1, double theta)` -- Changes m1 by multiplying on the right by a rotation matrix using rotation of y towards z. Theta is given in degrees.
- `rotate_zx(xform m1, double theta)` __ Changes m1 by multiplying on the right by a rotation matrix using rotation of z towards x. Theta is given in degrees.
- `world_to_camera(xform return, point eye, point at, vector up)` -- Using the given eye point, look at point and up vector, all expressed in world coordinates, returns the transformation from world to camera coordinates.
- `push(xform m1)` -- Pushes the given transform onto a stack of transforms.
- `pop(return xform)` -- Returns a transformation from the top of the transformation stack.
- `camera_to_clip(xform return, double fov, double near, double far, double aspect)` -- Returns a transformation to go from camera coordinates to normalized clipping coordinates (x, y, z all between 0 and 1). The vertical field of view of the camera is given by fov in degrees. Near and far are z distances of the near and far clipping planes respectively. Aspect is the ratio of horizontal to vertical size of the display device. This routine assumes that the viewing window is centered about the optical axis of the camera.
- `clip_to_device(xform return, int width, int height)` -- Returns a transformation to go from normalized clipping coordinates to device coordinates. Width and height are the size (in pixels) of the final viewing device.

Objects

- `draw_cube()` -- Run the faces of the cube (+/- 1 in x, y, and z) through the transformation pipeline.
- `draw_sphere()` -- Run a sphere (or some representation of a sphere) through the pipeline. The sphere should be centered at the origin and have a radius given by a parameter.
- `draw_cone()` -- Run a cone through the pipeline. The cone should have a circular base of a given radius on the xy plane. The cone should have a given height in the positive z direction.
- `draw_cylinder()` -- Run a circular cylinder through the pipeline. The ends should be circles of a given radius parallel to the xy plane. The circles are centered at (0, 0, zmin) and (0, 0, zmax) where zmin and zmax are the z coordinates of the ends of the cylinder. Thus the cylinder extends from -radius to radius in x and y and from zmin to zmax in z.
- `draw_disk()` -- Run a circular disc through the pipeline. The disk should have a given radius. The disk is parallel to the xy plane and centered about the z axis. A height parameters gives the position of the disk along the z axis.

Implementation

Your rendering engine will need a whole new group of global variables to keep track of. Following is a list of the information to be stored and the initial values that they should have.

- Transformation matrices for the object to world transformation (called the current transform), the world to clipping coordinates transformation, and the clipping to device coordinates transformation. These matrices do not have default values.
- You will need a stack of transformation matrices to hold the values of the current modeling transformation as it changes. This is *not* a stack where each stack entry holds one of the 16 values in a transformation matrix. Rather this is a stack where each entry is an entire 4 by 4 matrix. You may implement the stack any way you wish. A maximum stack depth of 10 to 30 should be sufficient.
- The camera field of view in degrees. Defaults to 90.
- Camera near clipping depth and camera far clipping depth. These should be initialized to 1.0 and 1.0e+09 respectively.
- A point for the camera eyepoint. Defaults to (0, 0, 0).
- A point for the camera look at point. Defaults to (0, 0, -1). Consequently, the default view is at the world origin looking down the negative z axis.

- A vector for the camera up vector. Defaults to (0, 1, 0) (positive y is up, positive x is to the right.)

You should implement two graphics pipelines, one for points and one for lines. The point pipeline should take a homogeneous point and transform it by the current transform and the world to clipping transform. At this stage the resulting point (in normalized clipping coordinates) should be tested to see if it lies in the canonical view volume (from 0 to 1 in x, y, and z). If it passes this test, the point is converted to device coordinates and plotted.

The pipeline for lines should take two arguments, a single homogeneous point and a flag that indicates whether the point should be moved to or drawn to. Regardless of the flag, the point should be transformed through object coordinates to world coordinates to clipping coordinates. At this step if the flag is set for moving only, the point should be stored (in clipping coordinates) in a static or global variable. If the line is to be drawn, then both the old_vertex (stored in the global variable) and the new vertex are transformed to device coordinates and a line drawn between the resulting coordinates. At the end of this process, the clipping coordinate version of the new point should be stored as the old vertex (in the global variable).

The following functionality should be added to the methods of the `REDirect` class.

`rd_world_begin()`
: The current transform should be set to the identity. The world to camera transformation can be computed using the camera eyepoint, look at point and up vector. The camera to clipping coordinate transformation matrix can be computed using the near and far clipping depths and the field of view. These last two transformations can be combined and stored as the world to clipping coordinate matrix. The clipping coordinate to device coordinate transform is also computed here.

`rd_point()`
: Takes a point, turns it into a homogeneous point and passes it to the point pipeline.

`rd_line()`
: Takes two points, converts them into homogeneous points and passes them to the line pipeline one at a time, moving to the first point and drawing to the second point.

`rd_translate()`
: Takes an array of three floats, creates a translation matrix and multiplies it by the current transform, storing the result back in the current transform.

`rd_scale()`
: Takes an array of three floats, the scale factors in x, y, and z, creates a scale matrix and multiplies it by the current transform, storing the result back in the current transform.

`rd_rotate_xy()`
: Takes a float which is the angle of rotation in degrees and creates a rotation matrix in the xy plane. The matrix is multiplied by the current transformation matrix and the results stored back in the current transform.

`rd_rotate_yz()`
: Ditto in the yz plane.

`rd_rotate_zx()`
: Same here.

`rd_camera_eye()`
: Store the values passed in into the global variable(s) set aside for this purpose.

`rd_camera_at()`
: Store in global variables.

`rd_camera_up()`
: Store globally.

`rd_camera_fov()`
: Store globally.

`rd_clipping()`
: Store the values in the near and far global clipping depths respectively.

`rd_pointset()`

This routine renders a set of points. It takes a string describing the information associated with each point, an array of numbers and the size of the array. The value of the string determines how many floating point values describe a point. This can be used to assign colors and surface normals to each point. Right now it should be ignored, assuming that there are only three values per point — x, y, and z.

The array gives the values assigned to the points -- in this case the x, y and z values of point 0, followed by the x, y, and z coordinates of point 1, and so on. The given size of this array (which has already been allocated when this function is called) is the number of points, not the number of values.

Each point in the set is passed through the point pipeline in turn, clipped, and plotted.

`rd_polyset()`

This routine renders a set of polygons. It takes a string describing the information associated with each polygon vertex, and two array of numbers and the size of those arrays. The value of the string determines how many floating point values describe a vertex. This can be used to assign colors, surface normals, and texture map coordinates to an object on a per vertex basis. Right now it should be ignored, assuming that there are only three values per vertex — x, y, and z.

The first array gives the values assigned to the vertices -- in this case the x, y and z values of vertex 0, followed by the x, y, and z coordinates of vertex 1, and so on. The given size of this array (which has already been allocated when this function is called) is the number of vertices, not the number of values.

The second array gives a list of vertices that describe the faces of the set. Faces in the array are separated by a value of -1. For example `1, 4, 7, 5, -1, 0, 1, 2, 3, -1 ...` means move to vertex 1, draw to vertex 4, draw to vertex 7, draw to vertex 5 and then back to vertex 1. Then move to vertex 0, draw to vertex 1, draw to vertices 2, 3, back to 0 and so on.

`rd_xform_push()`

Push a copy of the current transform onto the transformation stack. The current transformation is left unchanged.

`rd_xform_pop()`

Pop the top of the transformation stack into the current transform.

Input

Some more scene files are available that can be used to test your implementation. Some of the scene files also demonstrate the use of embedded objects (`ObjectBegin`, `ObjectEnd`, and `ObjectInstance`) to help build scenes. You do not have to implement anything to make use of embedded objects. They are automatically handled by the scene file parser.

Pass off

At pass off time, show your system running your version of the test scene files. Be prepared to modify the scene files and regenerate images during the pass off.

As always, make a scene of your own, and show the resulting image at passoff.

E-mail copies of your source code files, and a copy of the scene file for your original scene. Use the electronic submission method described in the course web pages.